Security - Hands-On 4
Cryptography
Prime Numbers

Esilv 2025

arnaud.nauwynck@gmail.com

Study on Prime-Number
... java code snippets

Step 1: Create a new Java (Maven) Project

typical dependencies to use
in pom.xml <dependencies>:

```
<dependencies>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <version>1.18.36</version>
        <scope>provided</scope>
    </dependency>

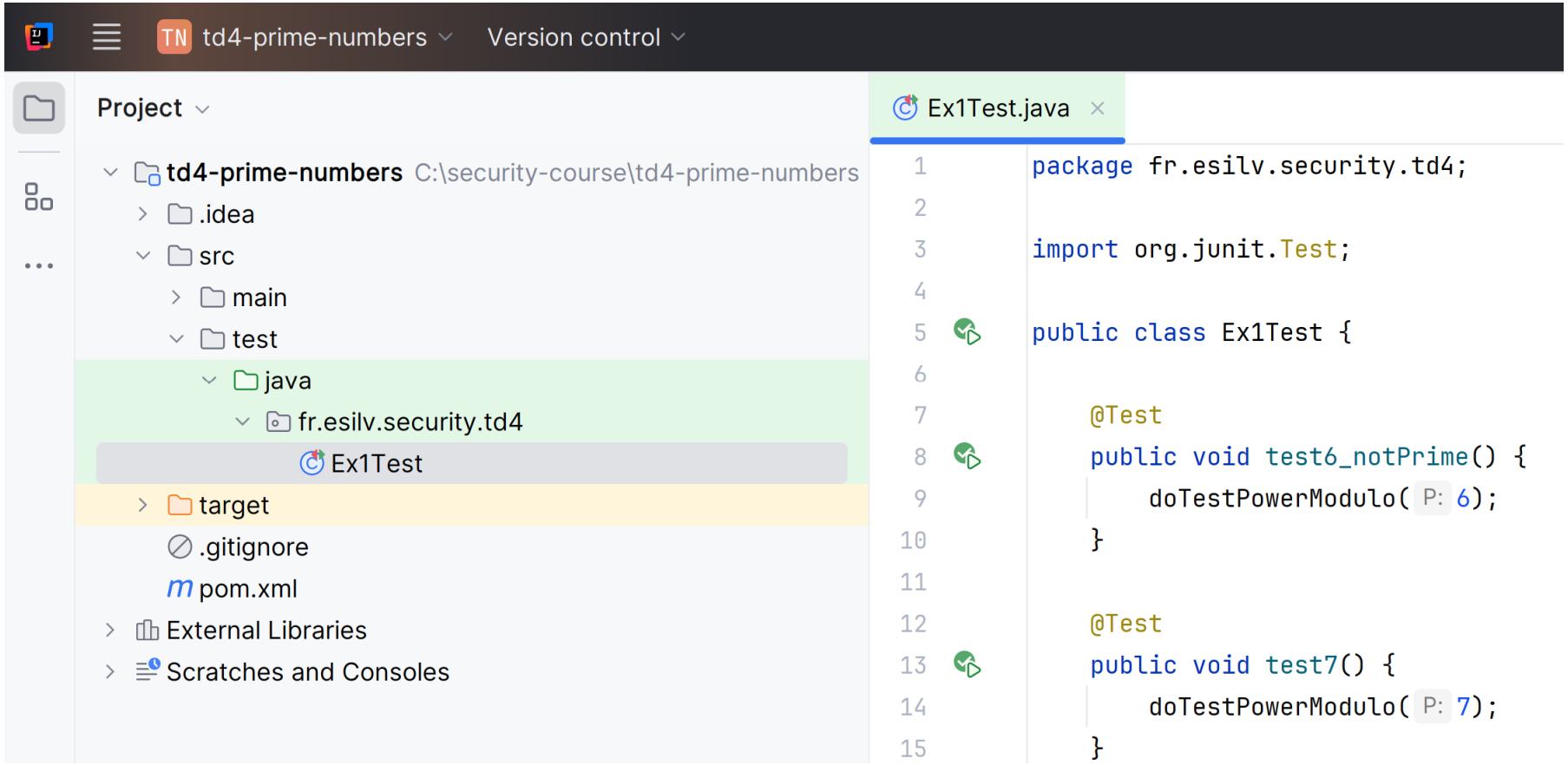
    <dependency>
        <artifactId>junit</artifactId>
        <groupId>junit</groupId>
        <version>4.13.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Ex1: Print consecutive powers of a number, modulo a prime number (ex: 101)

display the p first power values modulo P, for each element "i"
(warn to overflow, use modulo after each multiply)

- 2, 2^2%p, 2^3%p, 2^4%p ... 2^p%p
- 3, 3^2%p, 3^3%p, 3^4%p ... 3^p%p
- ...
- p-1, (p-1)^2, (p-1)^3, ...(p-1)^p%p

Run several samples as JUnit "@Test" methods for p=7, 11, 13, 17, 101, .. also run for p not prime, ex: 6




```
package fr.esilv.security.td4;

import org.junit.Test;

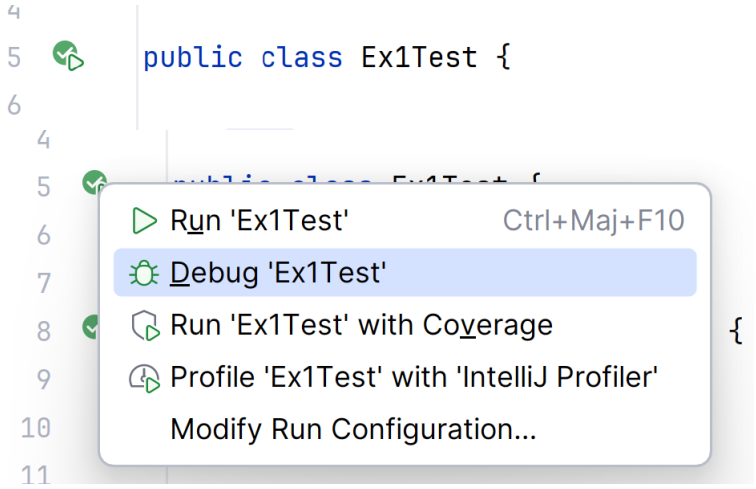
public class Ex1Test {

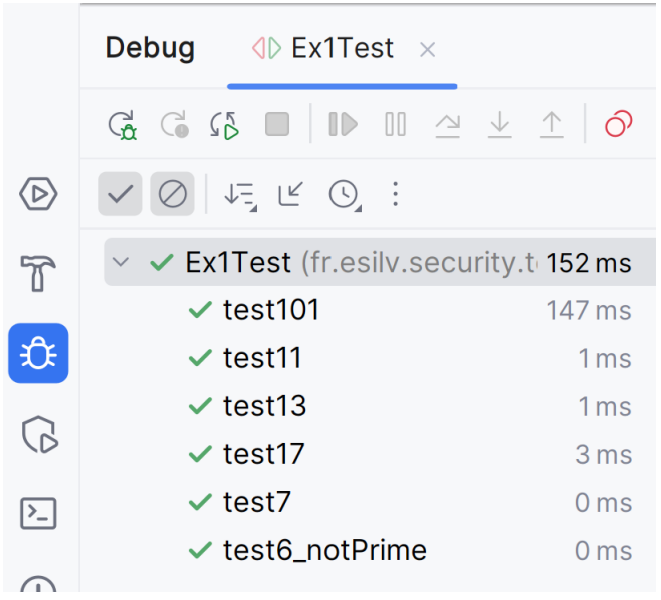
  @Test
  public void test6_notPrime() {
    doTestPowerModulo(6);
  }

  @Test
  public void test7() {
    doTestPowerModulo(7);
  }
  // ... complete code yourself
```

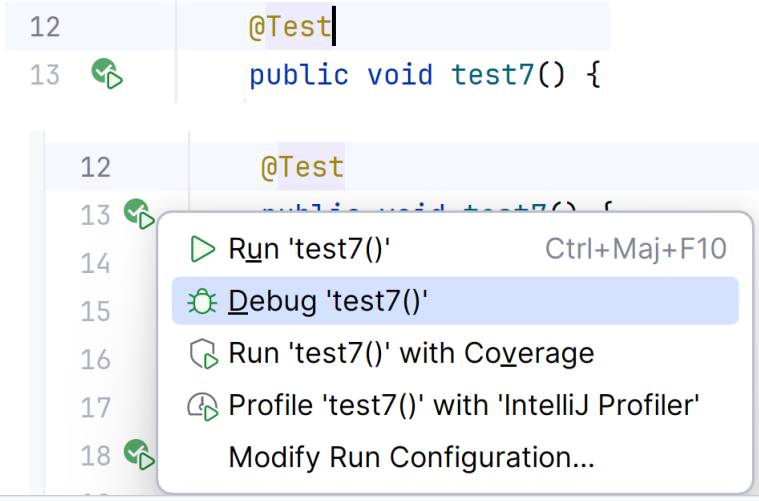
Run/Debug your Test(s)

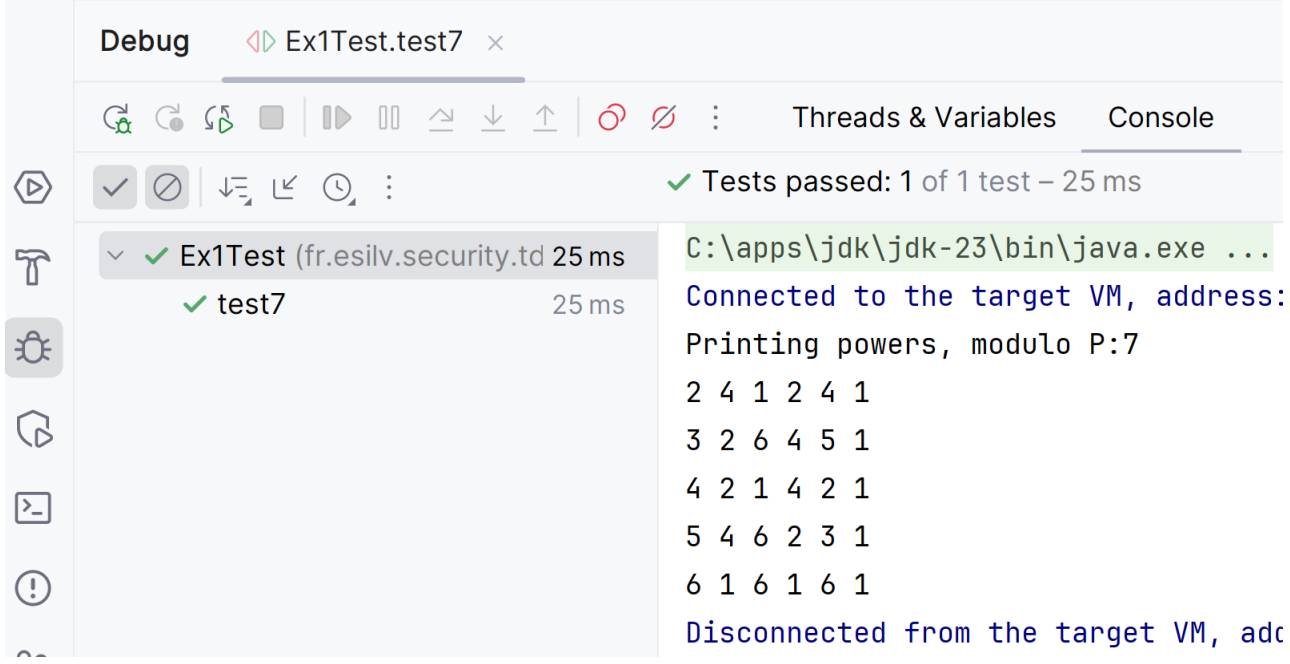
run all tests within class





run a specific @Test method





Ex 2 : How to efficiently raise number to powers 2, 4, 8, 16 … ?

Write a Test that raise n ^ 1048576   modulo P

1 048 576 = 1 mega (~1e6) = 1024*1024 = 2^20

How many multiplications and modulos used ?

(Optional) Ex 3: Write code to efficiently raise a number to power N="1000 000 000 000", with modulo "P"

do you need 1 trillions multiplications ?

so how many multiplications?

decompose N in base 2 (N = a0 + a1 . 2 + a2 2^2 + a3 2^3 + .. ak 2^k)

Ex 4 : code to use java BigInteger instead of int/long

cf built-in code "BigInteger.modPow(exponent, modulo)"

Returns a BigInteger whose value is $(this^{exponent} \bmod m)$. (Unlike `pow`, this method permits negative exponents.)

Params: `exponent` – the exponent.
`m` – the modulus.

Returns: $this^{exponent} \bmod m$

Throws: `ArithmeticException` – `m` ≤ 0 or the exponent is negative and this BigInteger is not *relatively prime* to `m`.

See Also: `modInverse`

```
@Contract(pure = true) @NotNull
2896 @  public BigInteger modPow(BigInteger exponent, BigInteger m) {
```

(Ex 4) Compute following:

p = 1000000000000000000000000000000000000000000000000000000000000001
a = 2000000000000000000000000000000000000000000000000000000000000001
m = 3000000000000000000000000000000000000000000000000000000000000001

a^p % m

Ex 5: Without iterating to check on divisors, proof that a given "P" is NOT prime

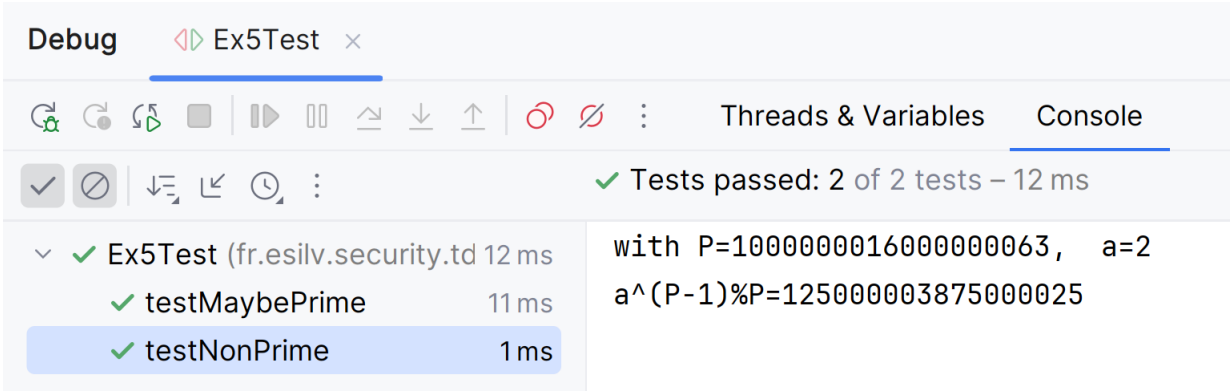
P = 10000000016000000063

Use the Fermat theorem. If P was prime, for all a => "a^(p-1) modulo P = 1";

test with choosen a =2, or a=3, .. or a=random

Use built-in method : BigInteger.modPow(exponent, modulo)

What is the probability that "a^(p-1) modulo P = 1" despite the fact that P were not prime ?



Notice that searching n1,n2 with N=n1*n2 is much more complex…

But when knowing, checking is trivial
n1=1000000007 x n2=1000000009

Ex5 : repeat primality test on n1, n2 compute 2^(n1-1) modulo n1

Ex5Test.java

```
7   public class Ex5Test {
43
44      @Test
45      public void testMaybePrime() {
46          System.out.println("with P=1000000007,  a=2");
47          final BigInteger P = BigInteger.valueOf(1000000007);
48          final BigInteger P_minus1 = P.add(BigInteger.valueOf(-1));
49          final BigInteger a2 = BigInteger.valueOf(2);
50          BigInteger ap1_p = a2.modPow(P_minus1, P);
51          System.out.println("a^(P-1)%P=" + ap1_p);
52      }
```

Debug Ex5Test

Threads & Variables Console

Tests passed: 2 of 2 tests – 12 ms

Ex5Test (fr.esilv.security.td 12 ms
testMaybePrime 11 ms
testNonPrime 1 ms

```
with P=1000000007,  a=2
a^(P-1)%P=1
```

Write the Fermat Primality test, with 5 internal number tries

(also call Miller-Rabin test)

See source java built-in source code: BigInteger.isProbablePrime(certainty)

using standard java.security.* package
(class MessageDigest, ..)

compare with bash command line tools

Ex 6: Use Java API to compute "SHA-256" of text (ex: "Hello World") print result in Hexa

```
@Test
public void testMessageDigest() throws Exception {
    byte[] data = "Hello World".getBytes();
    doTestSha256(data);
}
```

```
public void doTestSha256(byte[] data) throws Exception {
        // **** complete code yourself ****
        byte[] digest =  ****

        System.out.println("SHA-256 printed (binary, non asci)");
        System.out.println(new String(digest));

        String hex = HexFormat.of().formatHex(digest);
        System.out.println("SHA-256 printed (hex)");
        System.out.println(hex);
    }
```

Ex 7: compare with bash tool

```
$ echo -n "Hello World" | sha256sum
a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e *-

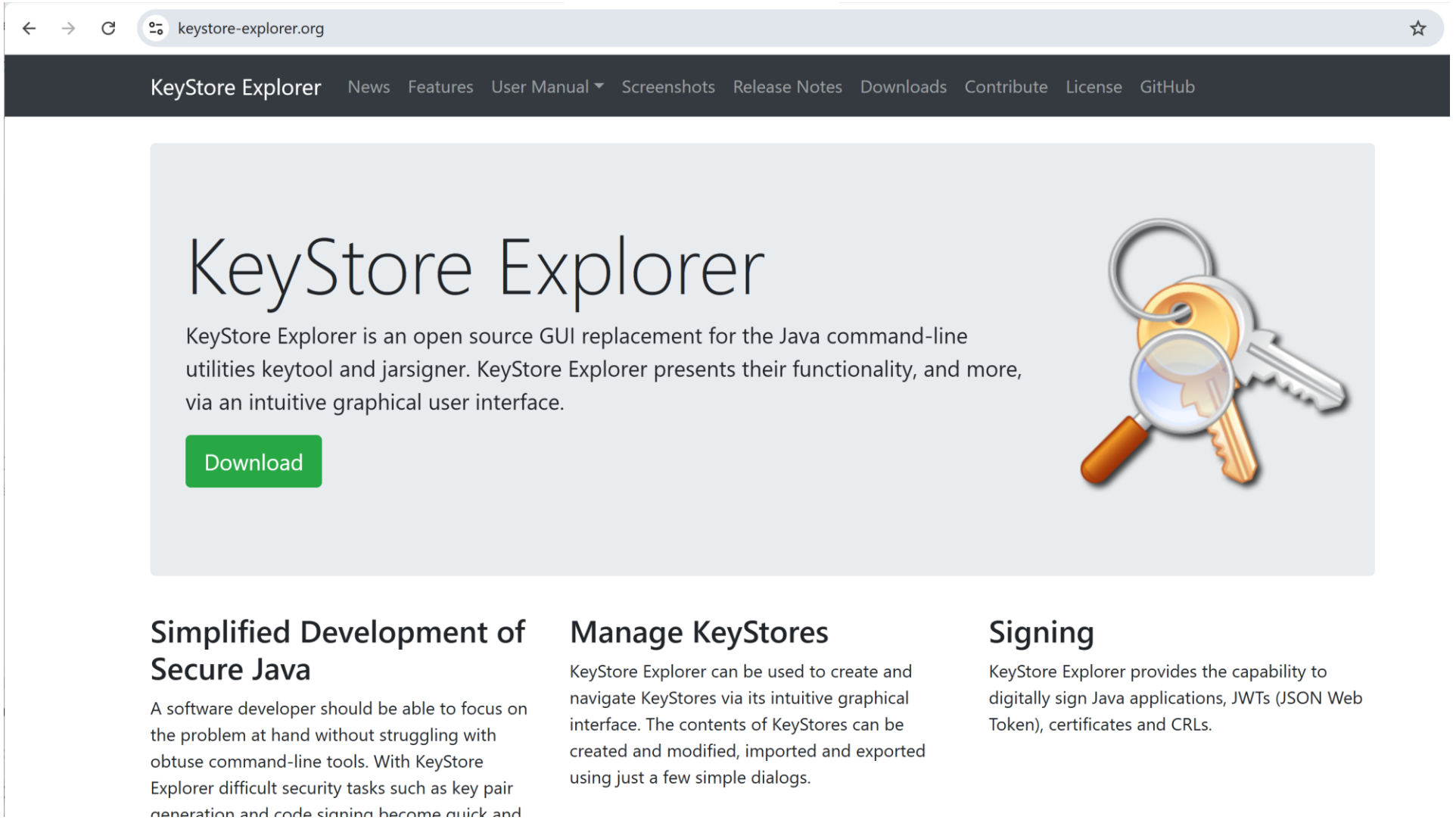
$ echo "Hello World" | sha256sum
d2a84f4b8b650937ec8f73cd8be2c74add5a911ba64df27458ed8229da804a26 *-
```

```
@Test
public void testMessageDigest() throws Exception {
    byte[] data = "Hello World".getBytes();
    doTestSha256(data);
}
@Test
public void testMessageDigest_newline() throws Exception {
    byte[] data = "Hello World\n".getBytes();
    doTestSha256(data);
}
```

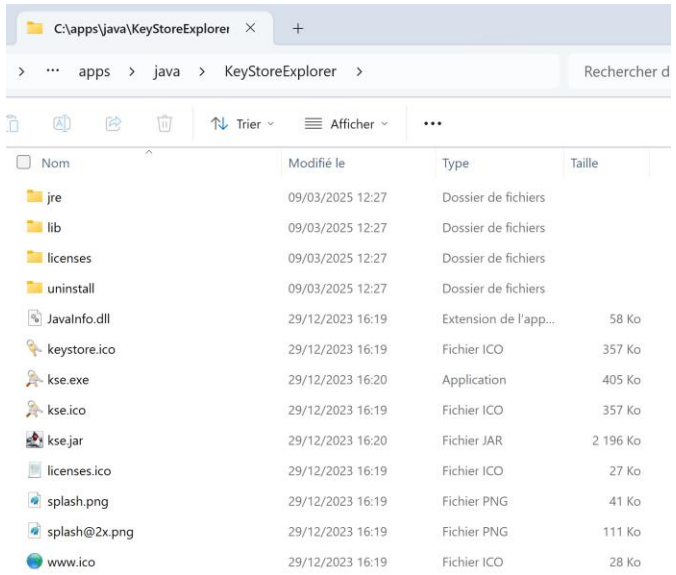
Ex 8: Use Java API to encrypt a text then decrypt, with RSA Assymetric algorithm

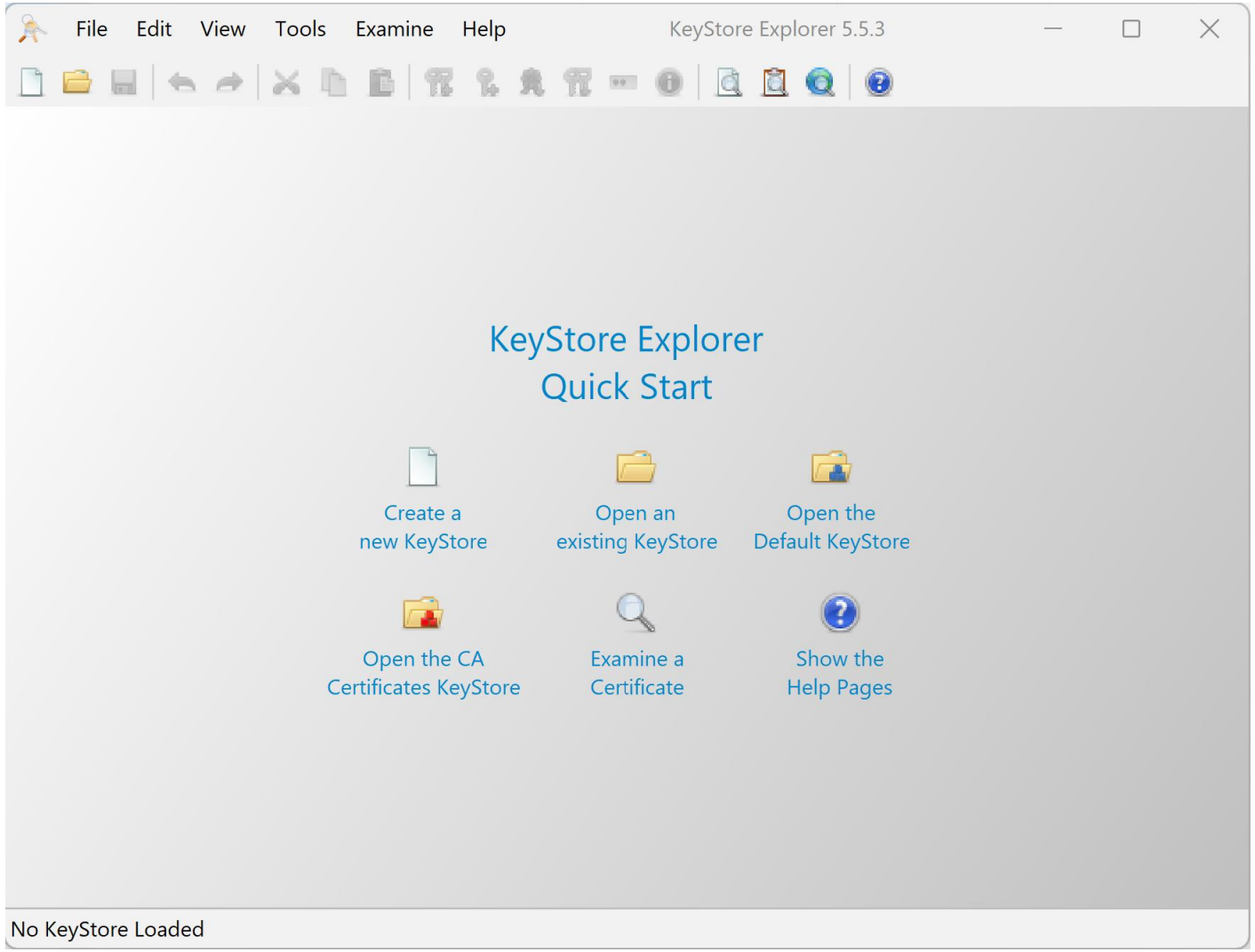
Ex 9: Use jdk/bin/keytool to generate a PrivateKey

Ex 10: Install java KeyStore Explorer (UI) https://keystore-explorer.org/

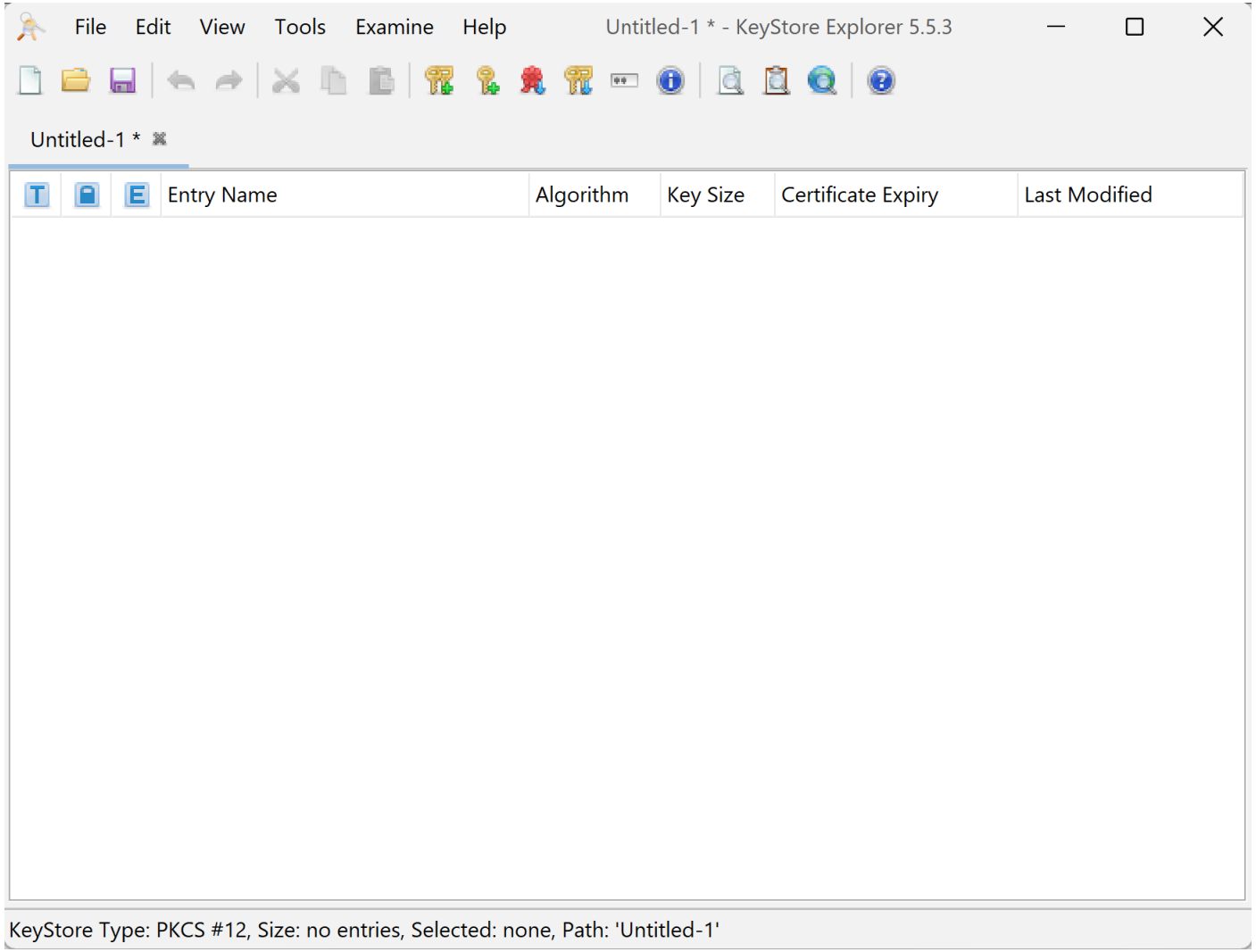


Launch "kse.exe"

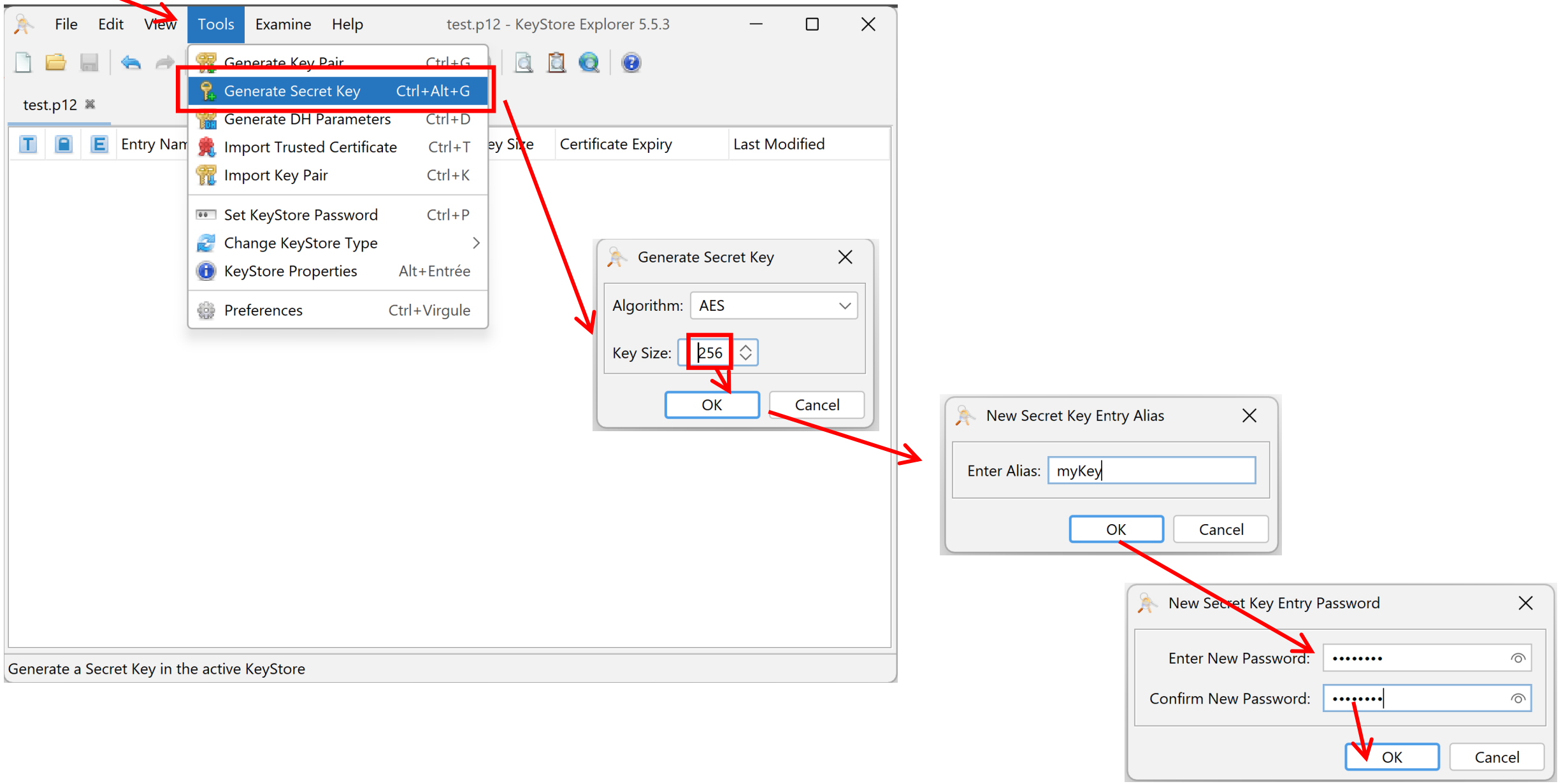




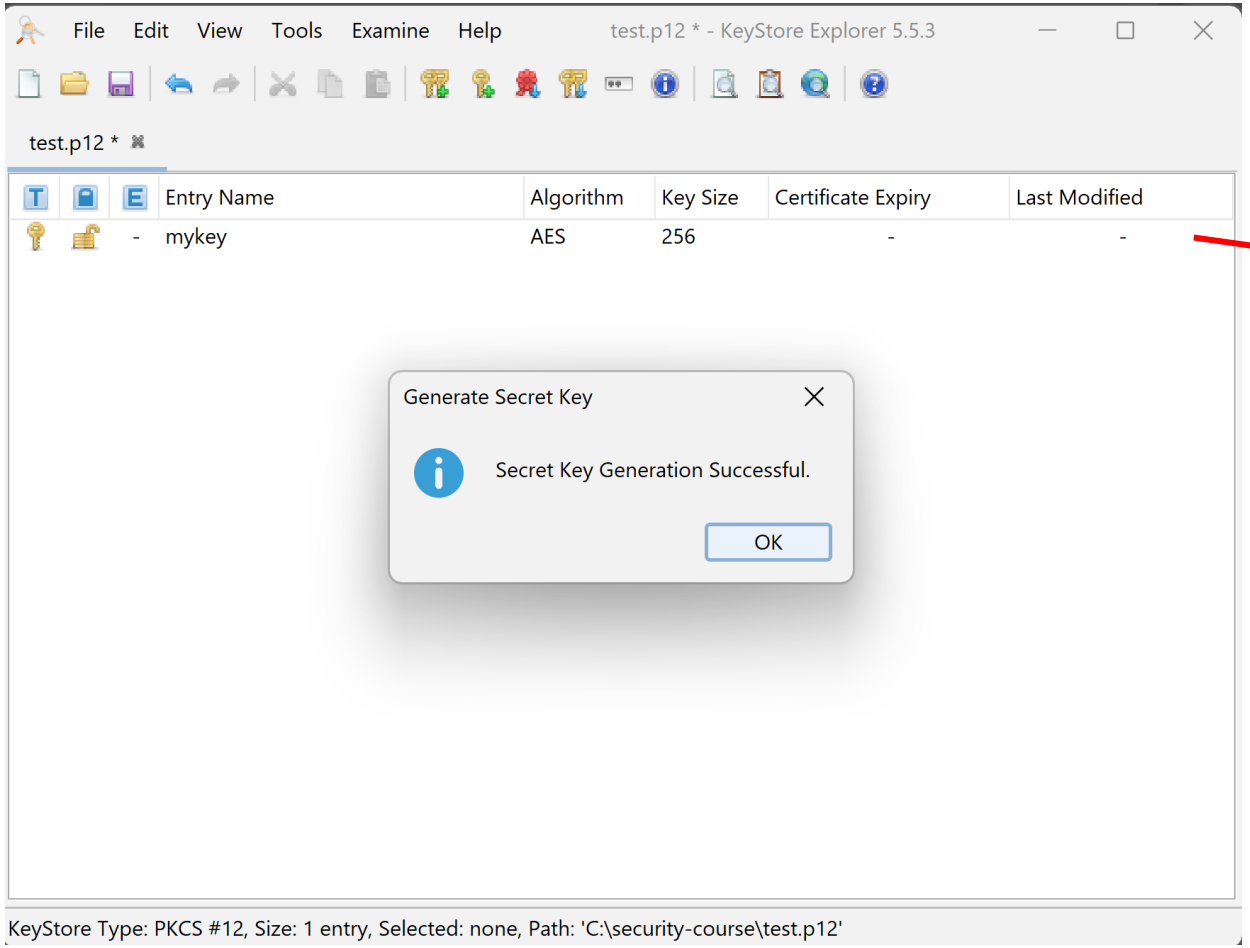
Create a new KeyStore
(or open an existing *.p12 / *.jks file)

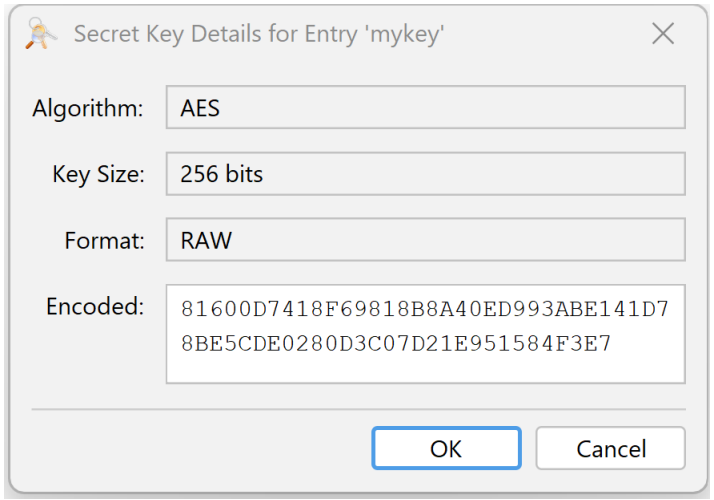


Generate a "Secret Key"



Created AES Key





Ex 11: compare with gpg

Cmder

```
$ gpg --help
gpg (GnuPG) 1.4.23
Copyright (C) 2015 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Home: ~/.gnupg
Supported algorithms:
Pubkey: RSA, RSA-E, RSA-S, ELG-E, DSA
Cipher: IDEA, 3DES, CAST5, BLOWFISH, AES, AES192, AES256, TWOFISH,
        CAMELLIA128, CAMELLIA192, CAMELLIA256
Hash: MD5, SHA1, RIPEMD160, SHA256, SHA384, SHA512, SHA224
Compression: Uncompressed, ZIP, ZLIB, BZIP2

Syntax: gpg [options] [files]
Sign, check, encrypt or decrypt
Default operation depends on the input data

Commands:

 -s, --sign [file]             make a signature
     --clearsign [file]        make a clear text signature
 -b, --detach-sign             make a detached signature
 -e, --encrypt                 encrypt data
 -c, --symmetric               encryption only with symmetric cipher
```

Ex 12: Use java.security.Cipher and KeyGenerator to encrypt a text then decrypt, with a Symmetric key algorithm

```
// encrypt then decrypt using symmetric key algorithm AES-256
@Test
public void testEncryptDecrypt() {
   String data = "Hello World".getBytes();
   String key = "1234567890123456";
   String iv = "1234567890123456";
   System.out.println("plainText: " + plainText + ", key: " + key + ", iv:" + iv);

   // encrypt
   byte[] encrypted = encryptAES(data, key, iv);
   System.out.println("encrypted: " + encrypted);

   // decrypt
   String decrypted = decryptAES(encrypted, key, iv);
   System.out.println("decrypted: " + decrypted);
}

private byte[] encryptAES(byte[] data, String key, String iv) {
   // ***** complete code yourself ******
   return null;
}
private String decryptAES(byte[] encrypted, String key, String iv) {
   // ***** complete code yourself *****
   return null;
}
```

Ex 13: repeat ex12, but use java class KeyStore to load the key from your keystore file

Question ?